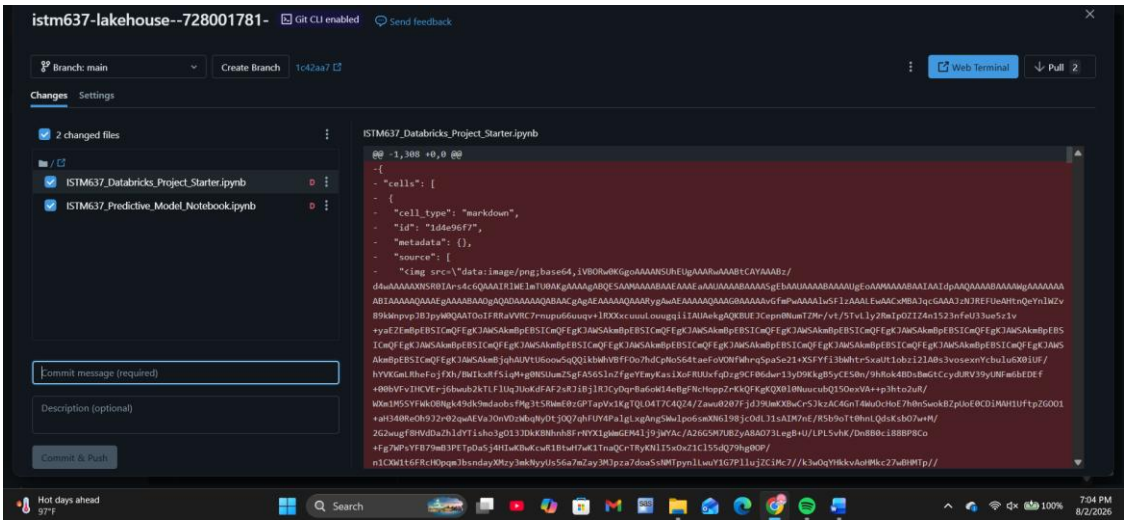
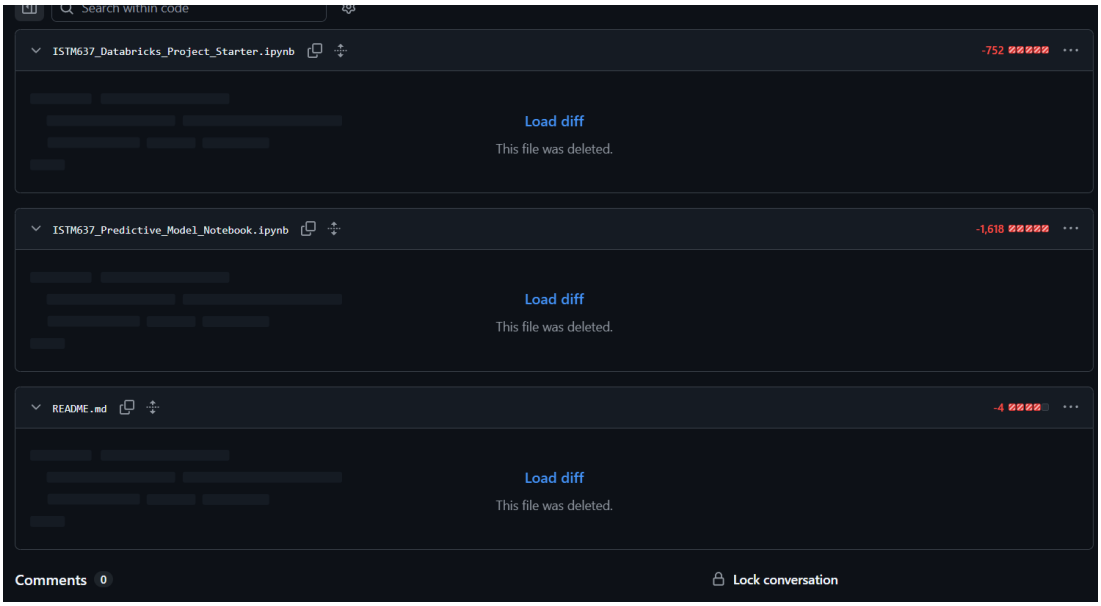
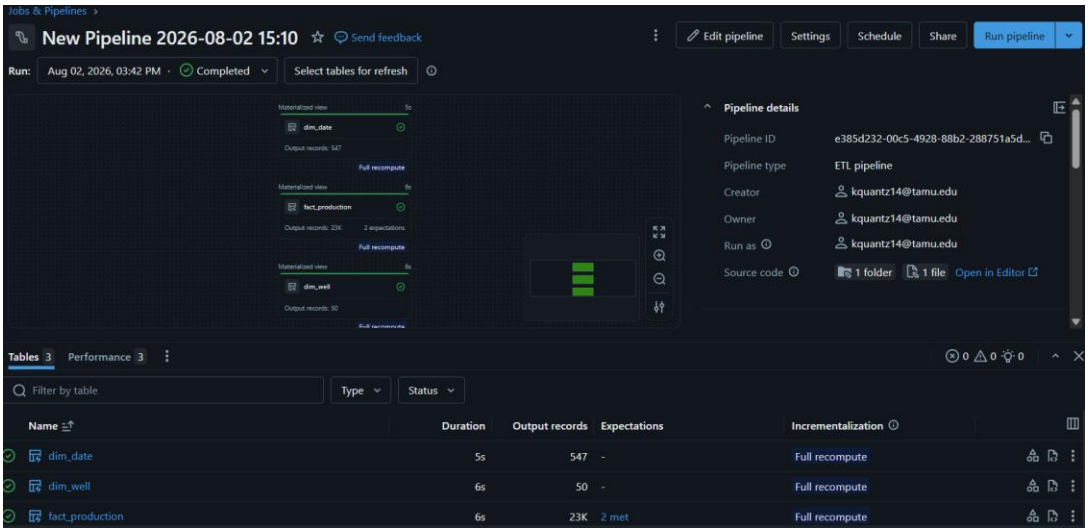
1.) Commit & Push was restricted and I used GitHub's web UI instead. I uploaded the files to Github, along with committing them because when I committed the files kept showing as "deleted" and I am unsure why. I uploaded the completed notebooks to GIT to be sure it contained the data.

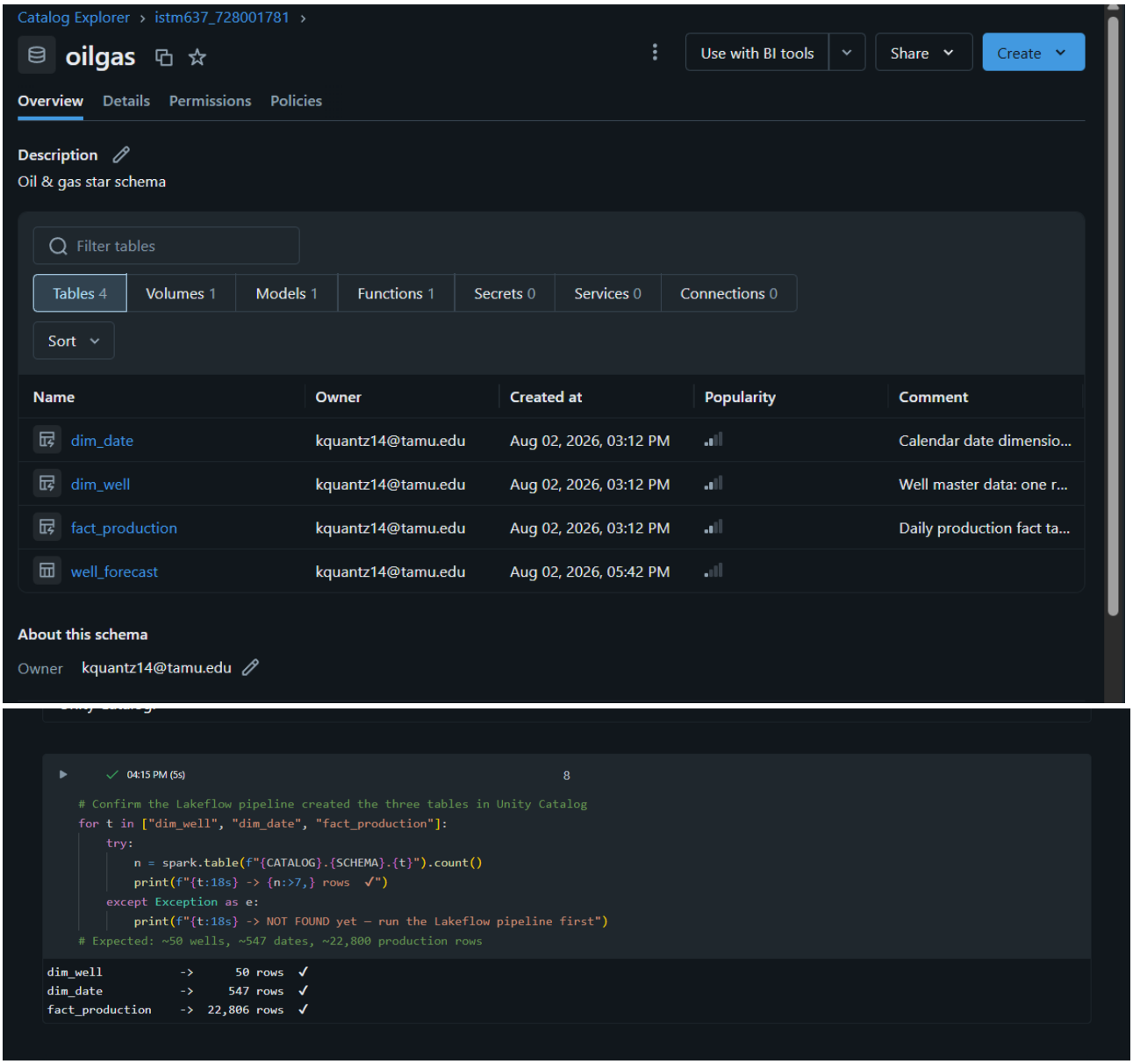




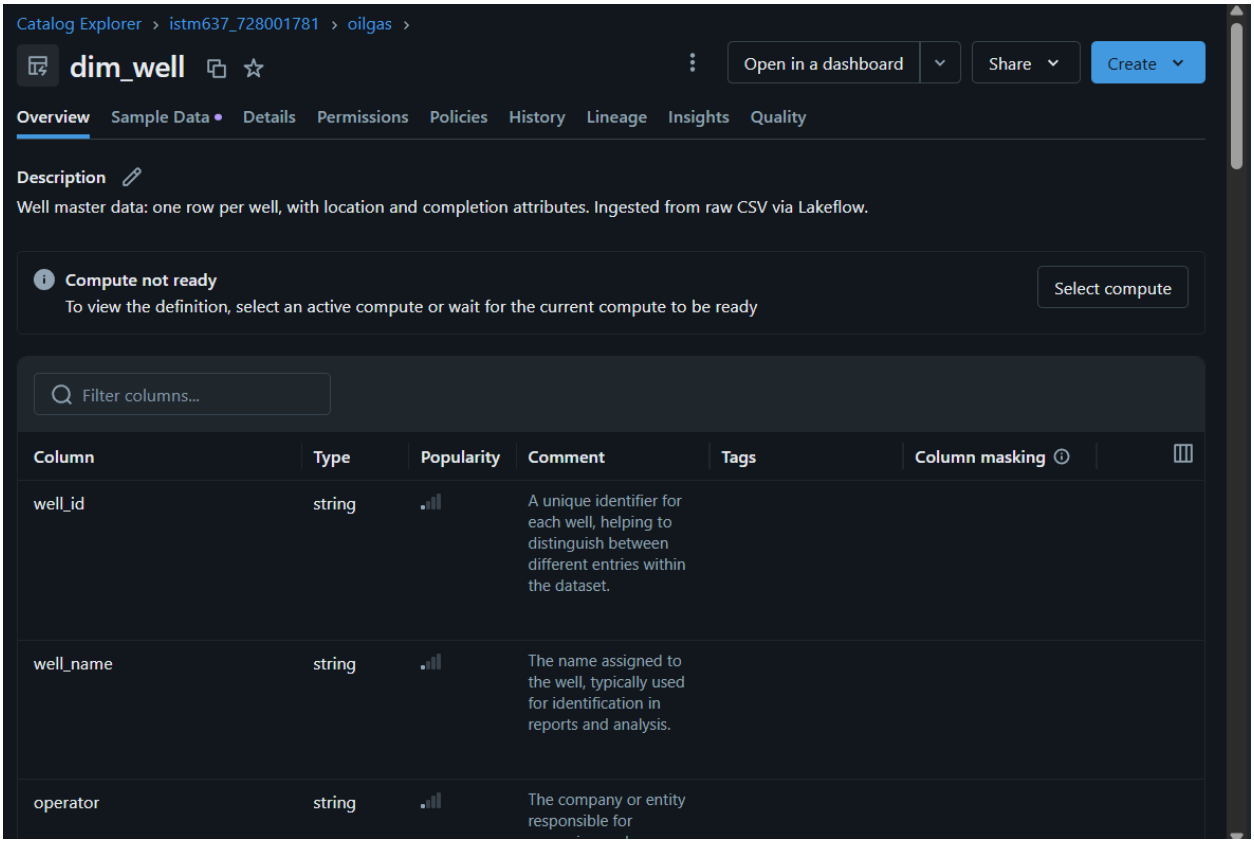
2.) I used the provided Lakeflow Declarative Pipeline (ISTM637_Lakeflow_Ingest_Pipeline.sql) to ingest the three CSV files from my raw volume into Unity Catalog. The pipeline run graph shows three materialized views—dim_well, dim_date, and fact_production, all completed successfully in my istm637_728001781.oilgas schema, with output counts of ~50 wells, ~547 dates, and ~22k production records, respectively. Catalog Explorer confirms these tables are registered in the oilgas schema alongside well_forecast, with appropriate descriptions for each. Finally,

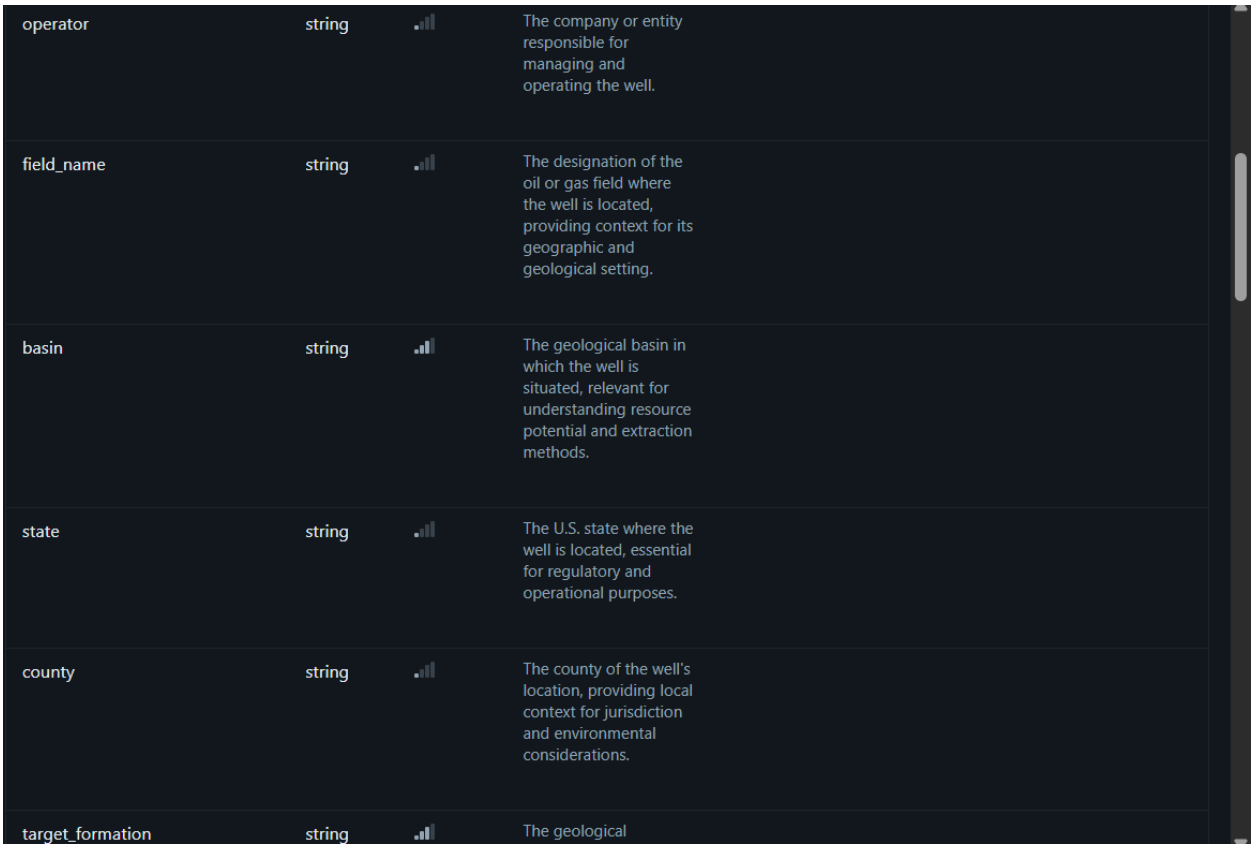
the verification cell in the starter notebook queries each table and prints dim_well -> 50 rows, dim_date -> 547 rows, and fact_production -> 22,806 rows, matching the expected star-schema grain of one row per well per producing day.





3.) AI Comments

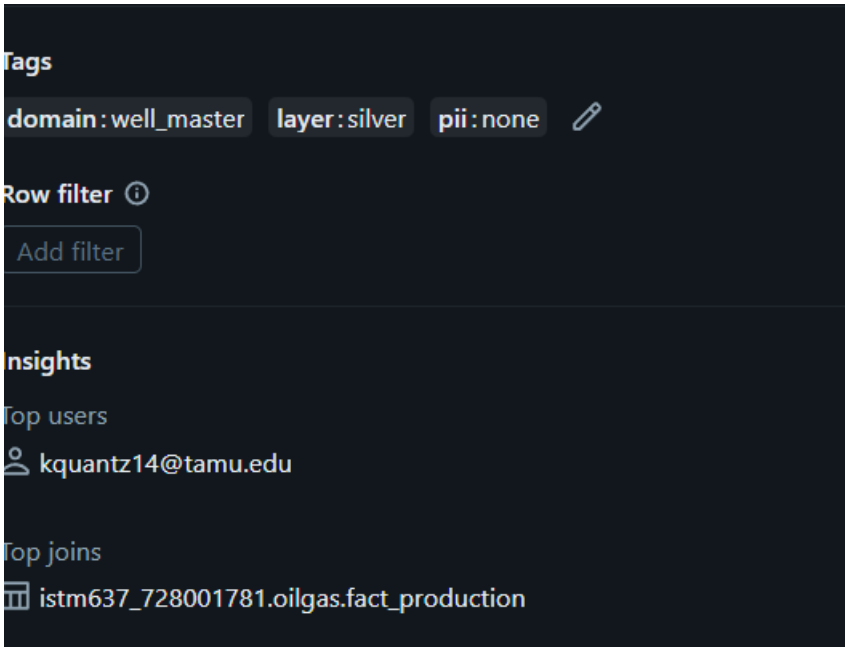

Catalog Explorer › istm637_728001781 › oilgas ›
dim_well
Open in a dashboard
Share
Create
Overview
Sample Data
Details
Permissions
Policies
History
Lineage
Insights
Quality
Description
Well master data: one row per well, with location and completion attributes. Ingested from raw CSV via Lakeflow.
Compute not ready
To view the definition, select an active compute or wait for the current compute to be ready
Select compute
Filter columns...
Column
Type
Popularity
Comment
Tags
Column masking
well_id
string
A unique identifier for each well, helping to distinguish between different entries within the dataset.
well_name
string
The name assigned to the well, typically used for identification in reports and analysis.
operator
string
The company or entity responsible for


operator
string
The company or entity responsible for managing and operating the well.
field_name
string
The designation of the oil or gas field where the well is located, providing context for its geographic and geological setting.
basin
string
The geological basin in which the well is situated, relevant for understanding resource potential and extraction methods.
state
string
The U.S. state where the well is located, essential for regulatory and operational purposes.
county
string
The county of the well's location, providing local context for jurisdiction and environmental considerations.
target_formation
string
The geological

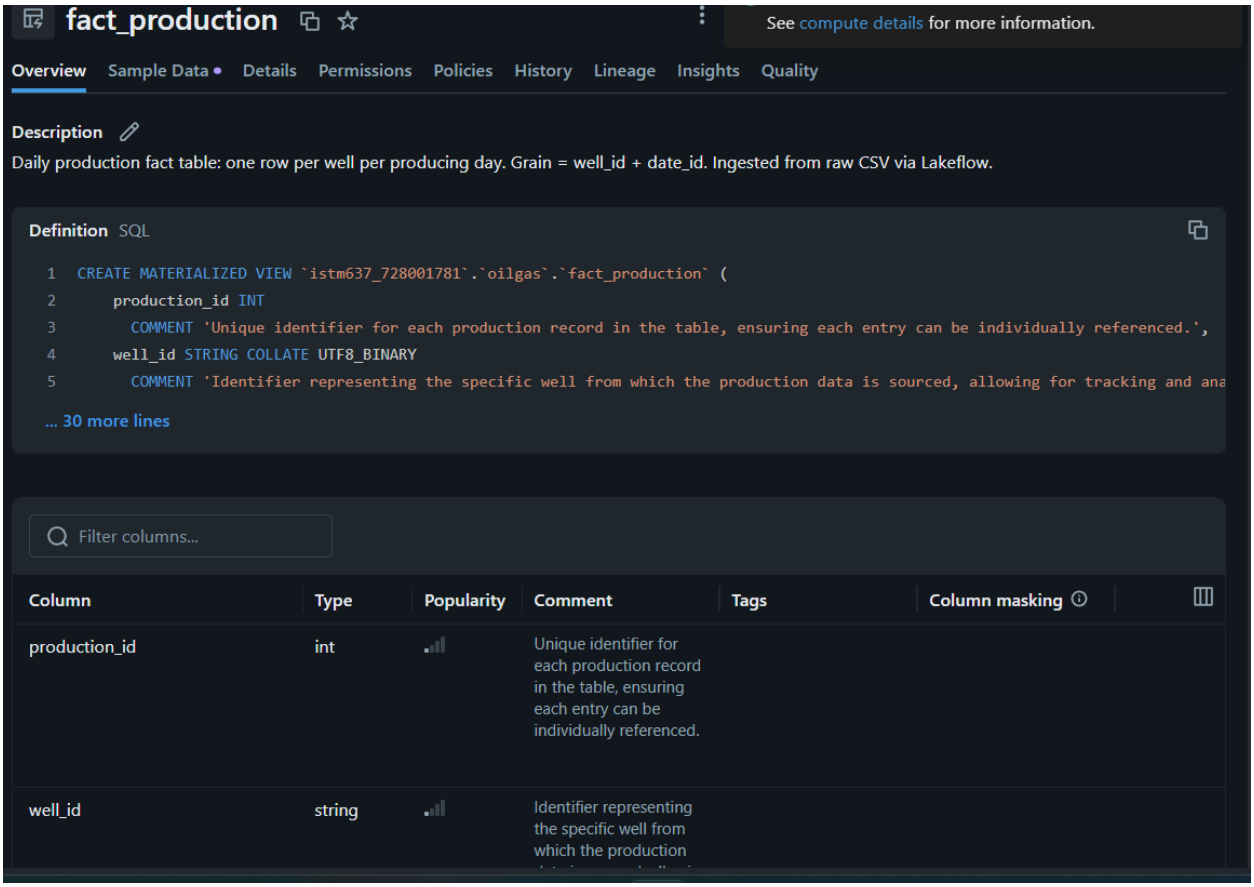
target_formation	string		The geological formation targeted by the well, indicating the specific rock strata that the well aims to exploit for oil or gas extraction.
well_type	string		Classification of the well, such as exploratory or production, that indicates its primary purpose.
lateral_length_ft	int		The length of the well's lateral section measured in feet, which impacts its production capability and extraction method.
latitude	double		The geographic coordinate indicating the north-south position of the well, useful for mapping and locating the site.
longitude	double		The geographic coordinate indicating the east-west position of the well, essential for accurate geolocation.

spud_date	date		The date when drilling operations for the well commenced, important for tracking development timelines.
completion_date	date		The date when the well was completed and ready for production, marking a significant milestone in its lifecycle.
status	string		Current operational status of the well, which can include classifications like active, inactive, or plugged.
initial_oil_potential_bopd	int		The estimated initial production capacity of the well in barrels of oil per day, providing insight into its potential profitability.
_rescued_data	string		Contains any data that may have been recovered or restored to address gaps or inconsistencies in the dataset.

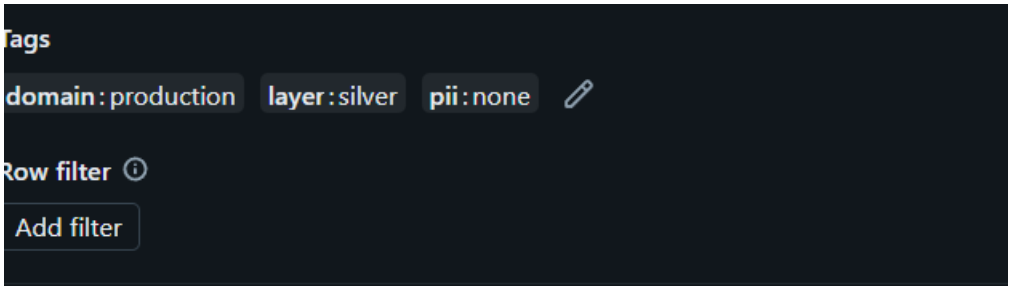
Tags:



Fact Table:



Tags:



The AI assistant's initial comments were helpful, but several were too generic for oil and gas production and needed refinement. I reviewed each suggestion and updated descriptions to use domain-specific terminology and units, for example clarifying that production rates are measured in barrels of oil per day (bopd) and that basin, field_name, and target_formation refer to distinct geological concepts rather than interchangeable location labels. I also adjusted operational fields like status and lateral_length_ft so the comments reflect the actual categories and units in the dataset, ensuring Genie can interpret the tables correctly when generating SQL.

4.)

Question	Genie correct?	Changed / fixed
Which basin produced the most oil overall?	Yes	No changes; existing metadata and joins were sufficient.
Show monthly oil production trend for 2024.	Yes	Added a metric to aggregate oil_bbl by month from dim_date.
What is the average water cut for wells in the Permian Basin?	No (initially)	Clarified “water cut” in instructions and column comments; re-run produced correct result.
Which operator has the most producing wells?	Yes	Added a trusted query so the correct producing-well filter was applied.
List the top 5 wells by total gas production.	No (initially)	Corrected the join between fact_production and dim_well; subsequent run was correct.

Test Questions:

Which basin produced the most oil overall?

› Thinking complete

The Permian Basin produced the most oil overall, with approximately **2.2 million barrels** of total production. [1]

The top five basins by total oil production are:

1. **Permian Basin**: 2,199,769 barrels
2. **Eagle Ford**: 2,148,451 barrels
3. **Delaware Basin**: 1,631,668 barrels
4. **DJ Basin**: 1,181,015 barrels
5. **Bakken**: 1,159,385 barrels

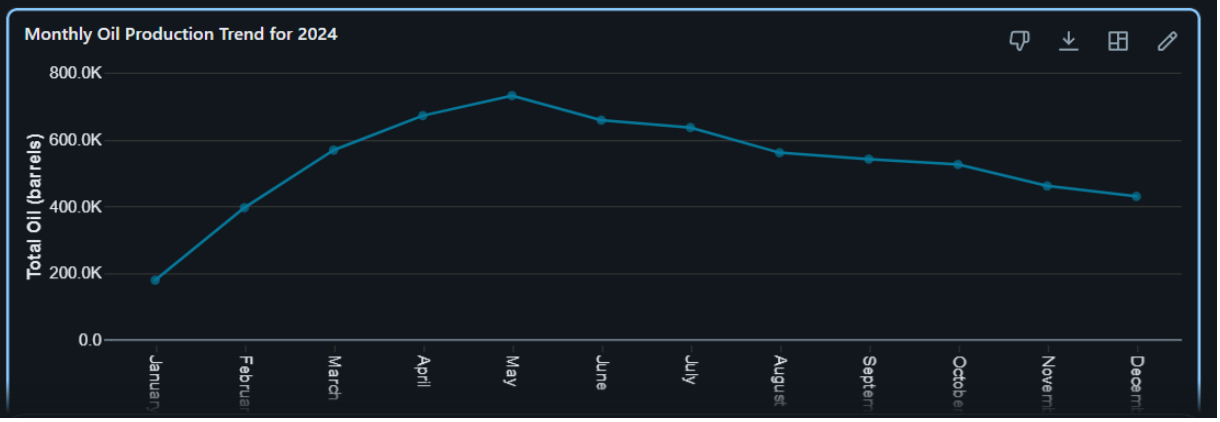
The Permian Basin leads by a narrow margin over Eagle Ford, with only about 51,000 barrels more in total production.

Show monthly oil_bbl trend for 2024.

› Thinking complete

Oil production in 2024 peaked in May at 731,000 barrels and declined steadily through the rest of the year. [1]

Show code



What is the average water cut for wells in the Permian Basin?

› Thinking complete

The average water cut for wells in the Permian Basin is 51.8%, meaning that water comprises approximately 52% of total liquid production (water + oil). [1]

This indicates that for every barrel of oil produced in the Permian Basin, slightly more than one barrel of water is also produced, which is typical for mature oil fields where water production increases over the well's lifecycle.

Which operator has the most producing wells?

› Thinking complete

Coterra and Marathon Oil are tied for the most producing wells, each operating 7 wells. [1]

The top operators by producing well count are:

1. **Coterra**: 7 wells
2. **Marathon Oil**: 7 wells
3. **EOG Resources**: 5 wells
4. **Occidental**: 5 wells
5. **ConocoPhillips**: 4 wells
6. **Diamondback**: 3 wells
7. **Devon Energy**: 3 wells
8. **Pioneer Natural**: 3 wells

List the top 5 wells by total gas production.

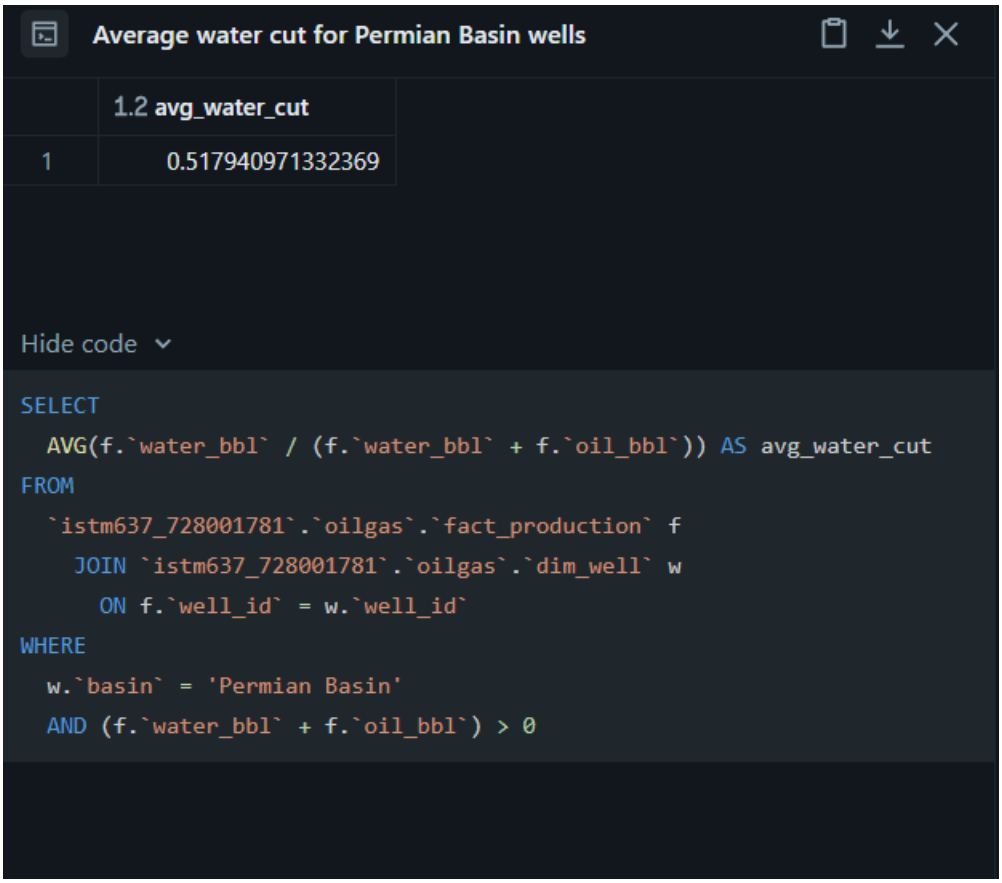
› Thinking complete

The top 5 wells by total gas production are: [1]

1. **Weld 35-34** (EOG Resources, DJ Basin, CO): 1,133,126 Mcf
2. **Lea 20-2** (Occidental, Delaware Basin, NM): 1,020,380 Mcf
3. **Martin 28-2H** (Coterra, Permian Basin, TX): 955,693 Mcf
4. **La Salle 5-11H** (Occidental, Eagle Ford, TX): 933,023 Mcf
5. **Eddy 25-15H** (ConocoPhillips, Delaware Basin, NM): 910,567 Mcf

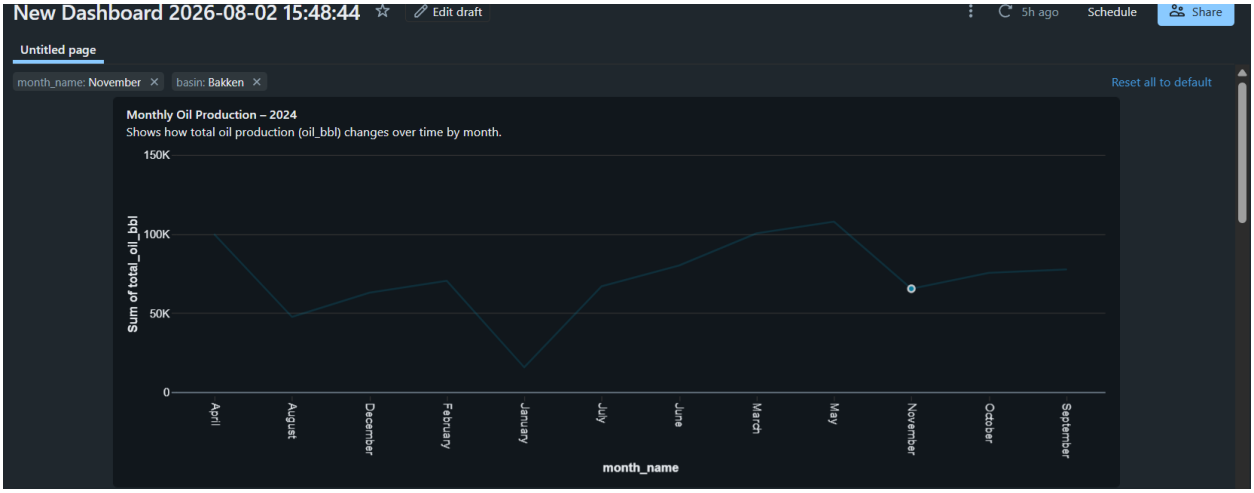
The leading well, Weld 35-34 in Colorado's DJ Basin, has produced over 1.1 billion cubic feet of natural gas, approximately 11% more than the second-place well.

SQL Code for 1 question:

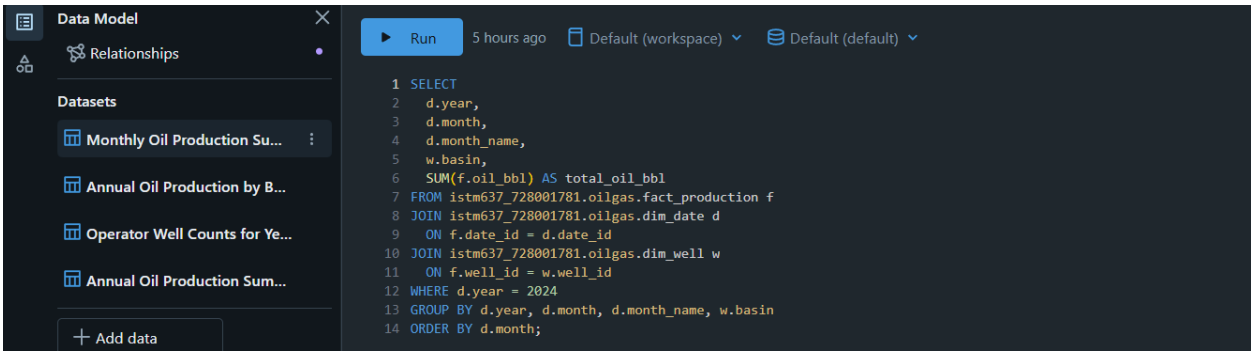


5.) Published dashboard showing all charts and before and after the filter applied

https://dbc-bdedc23f-47d9.cloud.databricks.com/dashboardsv3/01f18eb38dea17d3adaea2f238adbf3f/published?o=7474650324050514&f_e298ac28%7Eb6580941.s=Bakken&f_e298ac28%7Emonthly-oil-production-2024=%257B%2522columns%2522%253A%255B%2522x%2522%255D%252C%2522rows%2522%253A%255B%255B%2522November%2522%255D%255D%257D

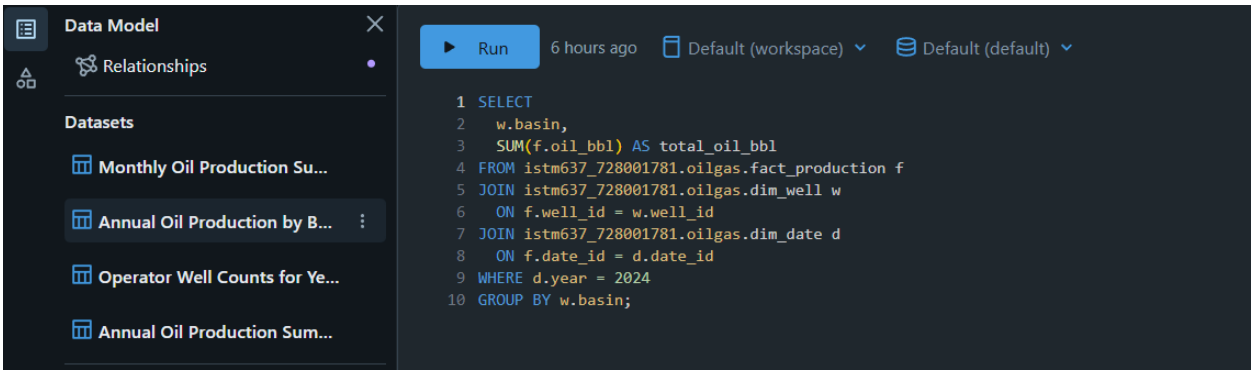

New Dashboard 2026-08-02 15:48:44
Edit draft
5h ago
Schedule
Share
Untitled page
month_name: November
basin: Bakken
Reset all to default
Monthly Oil Production – 2024
Shows how total oil production (oil_bbl) changes over time by month.
150K
100K
50K
0
Sum of total_oil_bbl
April
August
December
February
January
July
June
March
May
November
October
September
month_name

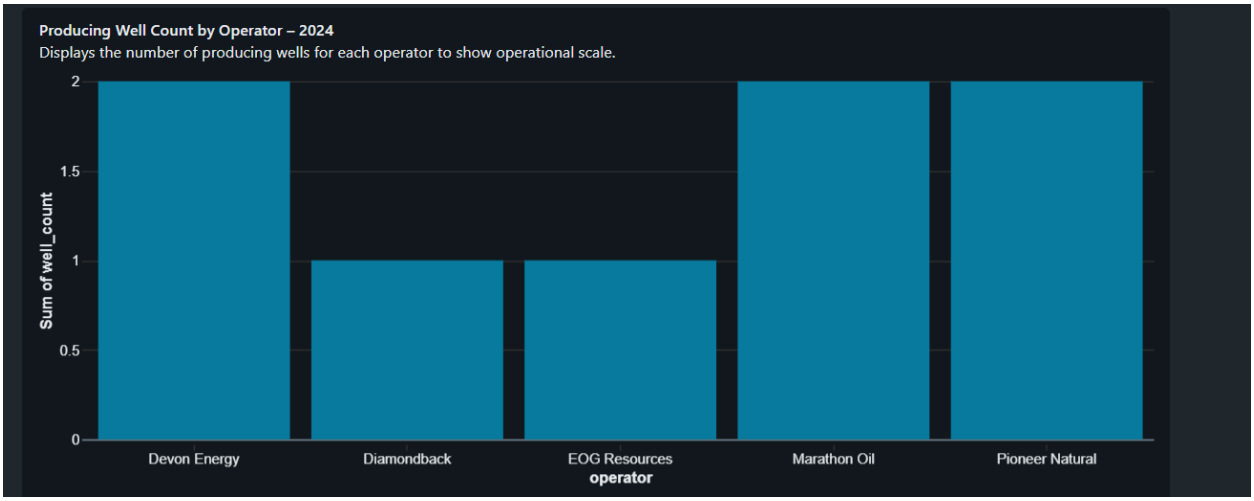
SQL query:


Data Model
Relationships
Datasets
Monthly Oil Production Su...
Annual Oil Production by B...
Operator Well Counts for Ye...
Annual Oil Production Sum...
Add data
Run
5 hours ago
Default (workspace)
Default (default)
1 SELECT
2 d.year,
3 d.month,
4 d.month_name,
5 w.basin,
6 SUM(f.oil_bbl) AS total_oil_bbl
7 FROM istm637_728001781.oilgas.fact_production f
8 JOIN istm637_728001781.oilgas.dim_date d
9 ON f.date_id = d.date_id
10 JOIN istm637_728001781.oilgas.dim_well w
11 ON f.well_id = w.well_id
12 WHERE d.year = 2024
13 GROUP BY d.year, d.month, d.month_name, w.basin
14 ORDER BY d.month;

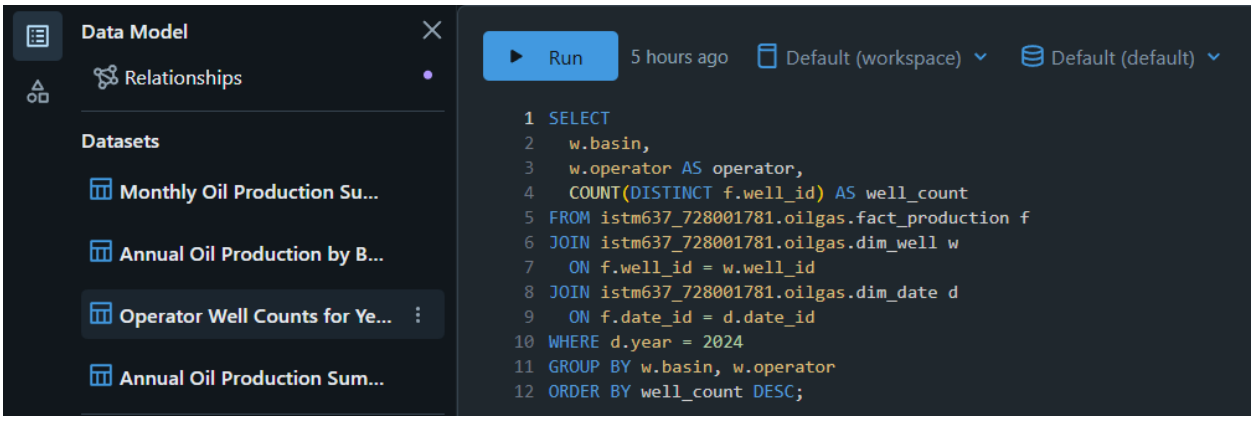

Oil Production by Basin – 2024
Compares total oil production across basins to highlight the highest-producing regions.
1M
800K
600K
400K
200K
0
Sum of total_oil_bbl
Bakken
basin

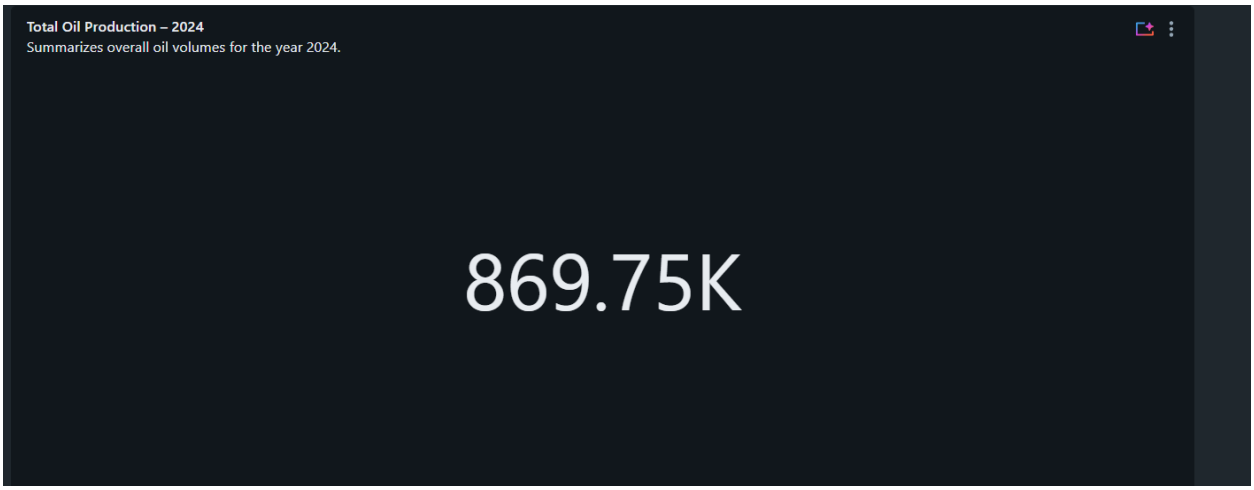
SQL Query:


Data Model
Relationships
Datasets
Monthly Oil Production Su...
Annual Oil Production by B...
Operator Well Counts for Ye...
Annual Oil Production Sum...
Run
6 hours ago
Default (workspace)
Default (default)
1 SELECT
2 w.basin,
3 SUM(f.oil_bbl) AS total_oil_bbl
4 FROM istm637_728001781.oilgas.fact_production f
5 JOIN istm637_728001781.oilgas.dim_well w
6 ON f.well_id = w.well_id
7 JOIN istm637_728001781.oilgas.dim_date d
8 ON f.date_id = d.date_id
9 WHERE d.year = 2024
10 GROUP BY w.basin;

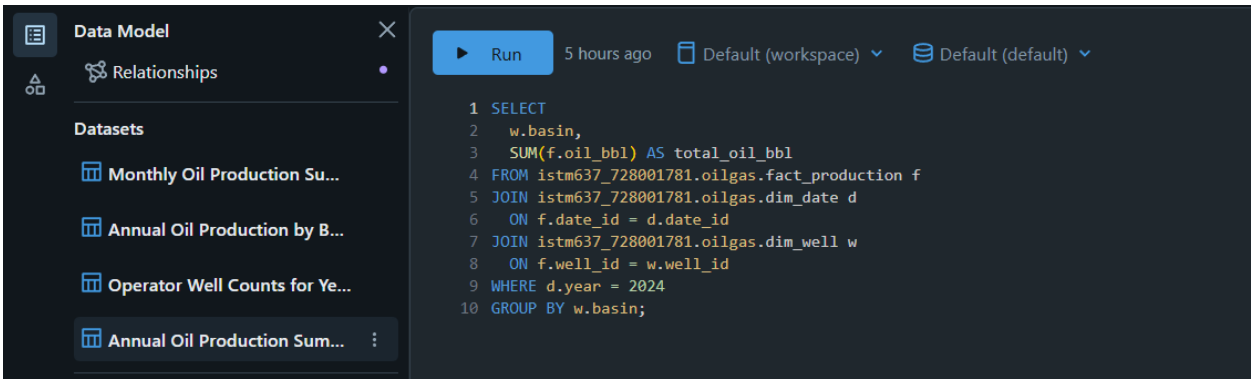

Producing Well Count by Operator – 2024
Displays the number of producing wells for each operator to show operational scale.
Sum of well_count
2
1.5
1
0.5
0
Devon Energy
Diamondback
EOG Resources
Marathon Oil
Pioneer Natural
operator

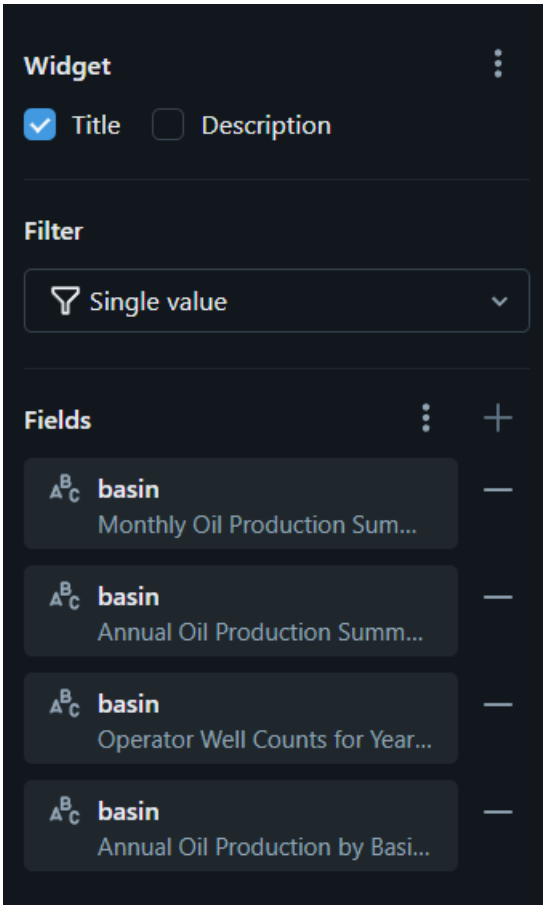
SQL Query:

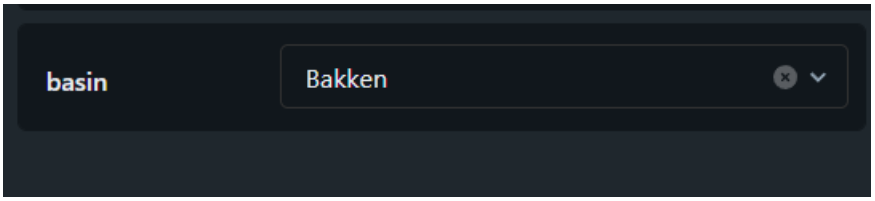




SQL Query:

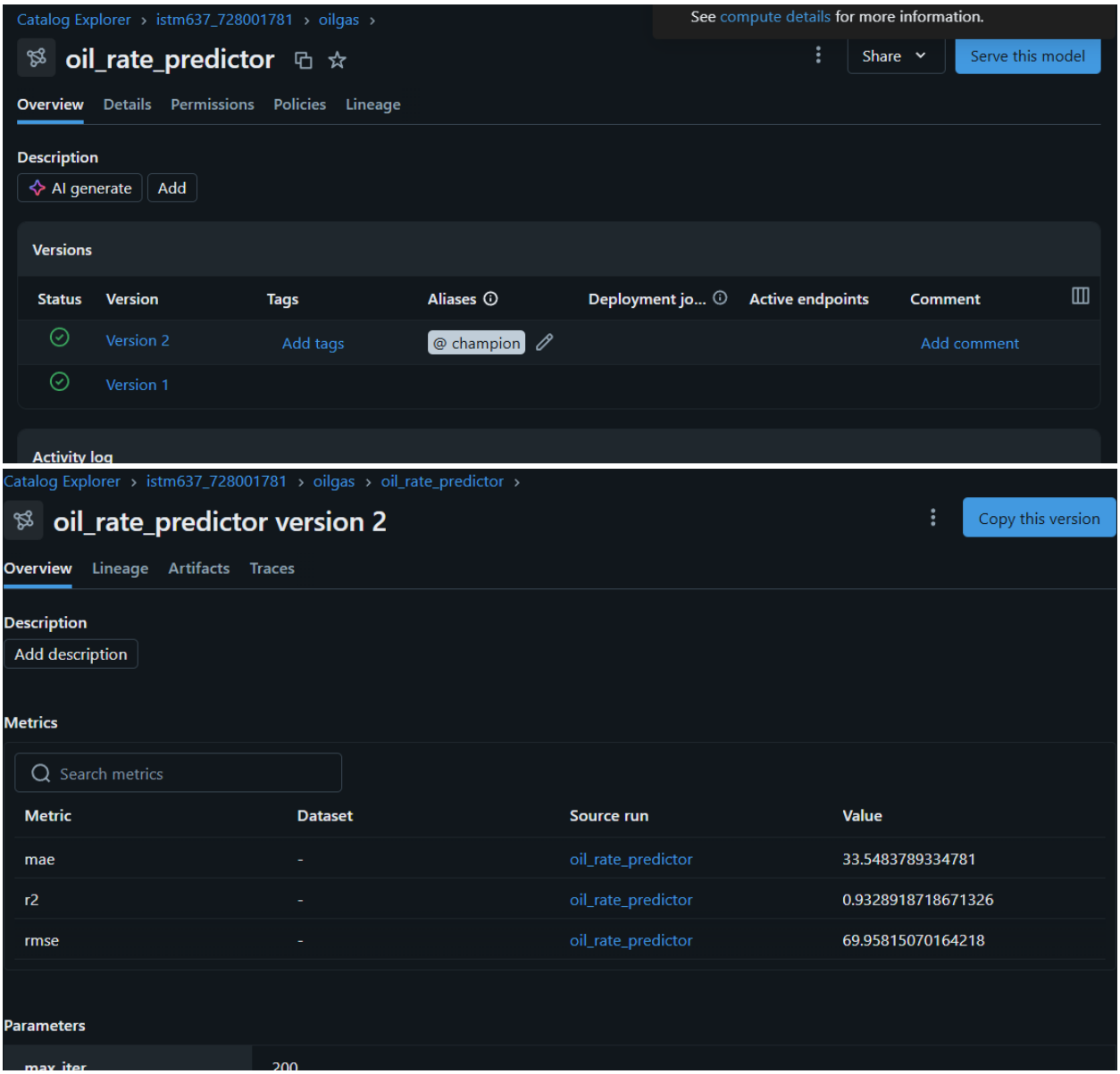




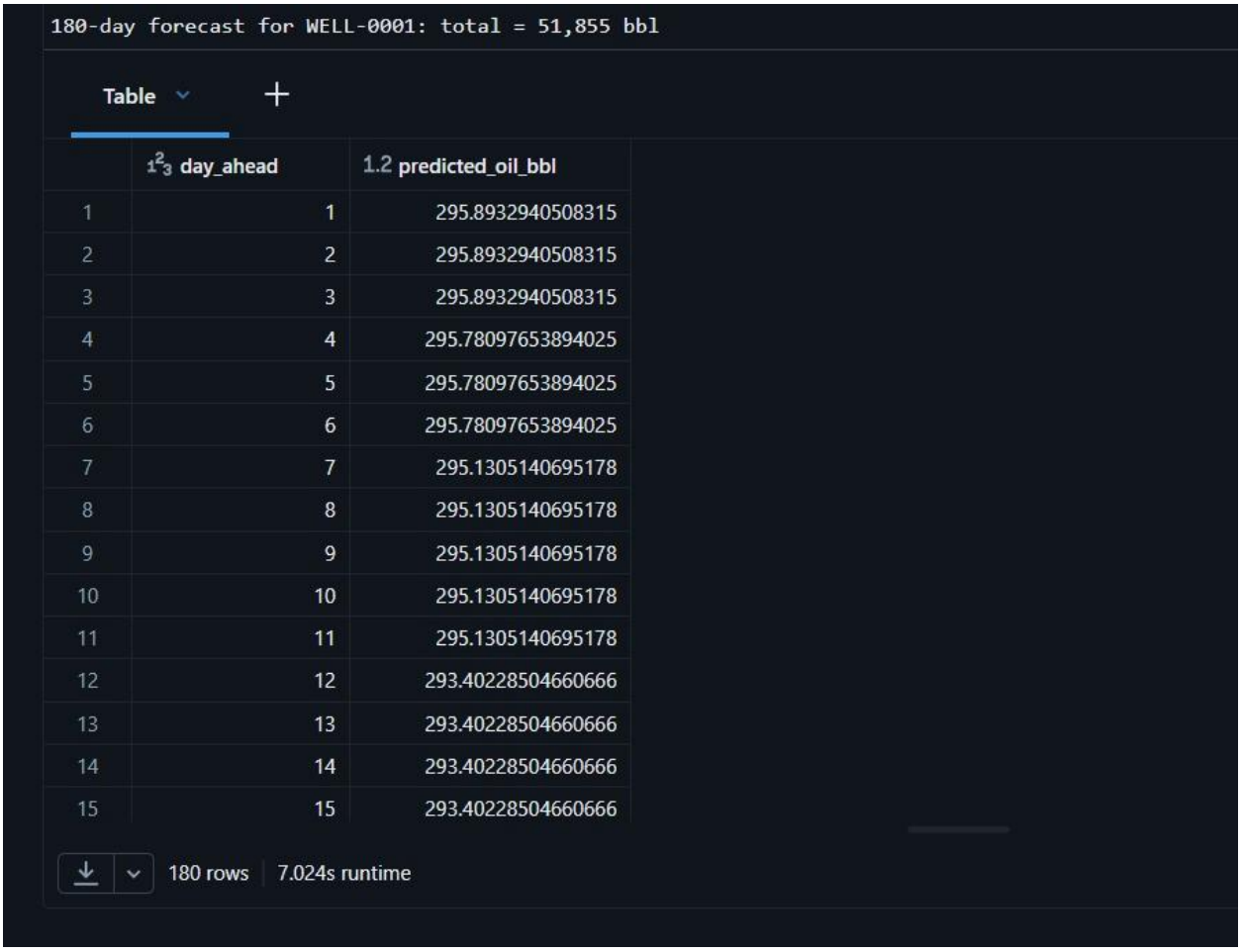


6.) MAE / RMSE /R-squared

Registered Model Screenshot



180 Day forecast for well-0001: Figure X shows the model's 180-day forecast for WELL-0001, with daily predicted oil_bbl and a total forecast of 51,855 bbl.



day_ahead	predicted_oil_bbl
1	295.8932940508315
2	295.8932940508315
3	295.8932940508315
4	295.78097653894025
5	295.78097653894025
6	295.78097653894025
7	295.1305140695178
8	295.1305140695178
9	295.1305140695178
10	295.1305140695178

11	295.1305140695178
12	293.40228504660666
13	293.40228504660666
14	293.40228504660666
15	293.40228504660666
16	293.40228504660666
17	293.40228504660666
18	293.40228504660666
19	293.40228504660666
20	293.40228504660666
21	293.40228504660666
22	293.40228504660666
23	293.40228504660666
24	293.40228504660666
25	293.40228504660666
26	293.40228504660666
27	294.471494192367
28	294.471494192367
29	294.471494192367
30	294.471494192367
31	294.471494192367
32	294.471494192367
33	294.471494192367
34	294.471494192367
35	294.471494192367
36	294.471494192367

37	294.471494192367
38	294.471494192367
39	294.471494192367
40	294.471494192367
41	290.1923140982846
42	290.1923140982846
43	290.1923140982846
44	290.1923140982846
45	290.1923140982846
46	290.1923140982846
47	290.686427664526
48	290.686427664526
49	290.686427664526
50	290.686427664526
51	290.686427664526
52	290.686427664526
53	290.686427664526
54	290.686427664526
55	290.686427664526
56	290.686427664526
57	290.0664973528937
58	290.0664973528937
59	290.0664973528937
60	290.0664973528937
61	290.0664973528937
62	290.0664973528937

63	290.0664973528937
64	290.0664973528937
65	290.0664973528937
66	290.0664973528937
67	290.0664973528937
68	290.0664973528937
69	290.0664973528937
70	290.0664973528937
71	290.0664973528937
72	290.0664973528937
73	290.0664973528937
74	290.0664973528937
75	290.0664973528937
76	290.0664973528937
77	290.0664973528937
78	289.04366079728413
79	289.04366079728413
80	289.04366079728413
81	289.04366079728413
82	289.04366079728413
83	289.04366079728413
84	289.04366079728413
85	289.04366079728413
86	289.04366079728413
87	284.86317340245614
88	284.86317340245614

89	284.86317340245614
90	284.86317340245614
91	284.86317340245614
92	284.4394542905666
93	284.4394542905666
94	284.4394542905666
95	284.4394542905666
96	284.4394542905666
97	284.4394542905666
98	284.4394542905666
99	284.4394542905666
100	284.4394542905666
101	284.4394542905666
102	284.4394542905666
103	284.4394542905666
104	284.4394542905666
105	284.4394542905666
106	284.4394542905666
107	284.4394542905666
108	284.4394542905666
109	284.4394542905666
110	284.4394542905666
111	284.4394542905666
112	284.4394542905666
113	284.4394542905666
114	284.4394542905666

115	284.4394542905666
116	284.4394542905666
117	284.4394542905666
118	284.4394542905666
119	284.4394542905666
120	284.4394542905666
121	284.4394542905666
122	284.4394542905666
123	284.4394542905666
124	284.4394542905666
125	284.4394542905666
126	284.4394542905666
127	284.4394542905666
128	284.4394542905666
129	284.4394542905666
130	284.4394542905666
131	284.4394542905666
132	284.4394542905666
133	284.4394542905666
134	284.4394542905666
135	284.4394542905666
136	284.4394542905666
137	284.4394542905666
138	284.4394542905666
139	284.4394542905666
140	284.4394542905666

141	284.4394542905666
142	284.4394542905666
143	284.4394542905666
144	284.4394542905666
145	284.4394542905666
146	284.4394542905666
147	284.4394542905666
148	284.4394542905666
149	284.4394542905666
150	284.4394542905666
151	284.4394542905666
152	284.4394542905666
153	284.4394542905666
154	284.4394542905666
155	284.4394542905666
156	284.4394542905666
157	284.4394542905666
158	284.4394542905666
159	284.4394542905666
160	284.4394542905666
161	284.4394542905666
162	284.4394542905666
163	284.4394542905666
164	284.4394542905666
165	284.4394542905666
166	284.4394542905666

167 284.4394542905666

168 284.4394542905666

169 284.4394542905666

170 284.4394542905666

171 284.4394542905666

172 284.4394542905666

173 284.4394542905666

174 284.4394542905666

175 284.4394542905666

176 284.4394542905666

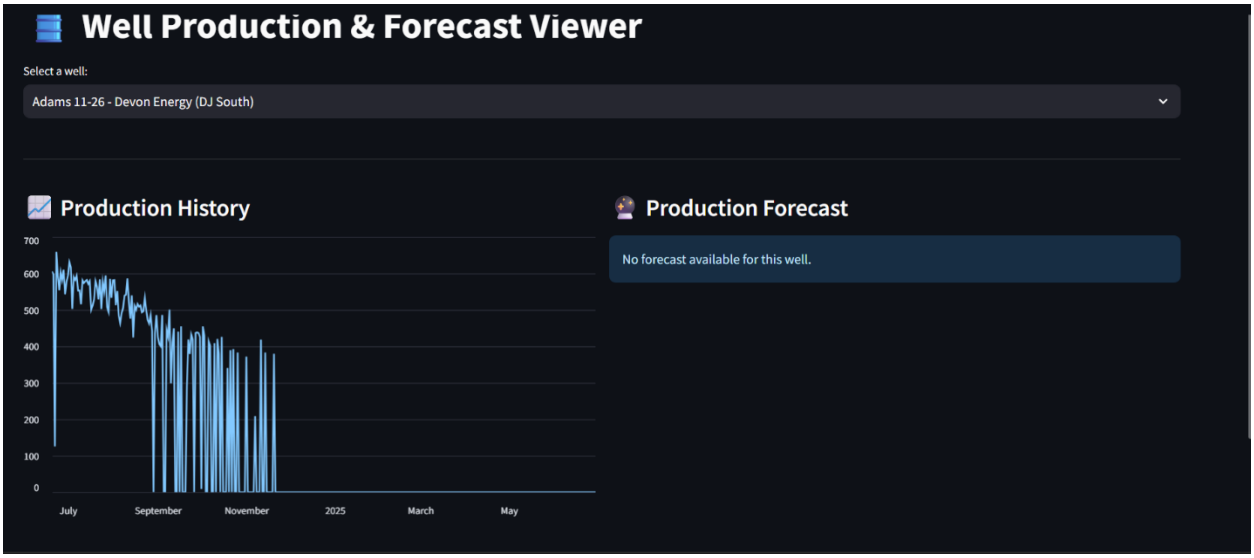
177 284.4394542905666

178 284.4394542905666

179 284.4394542905666

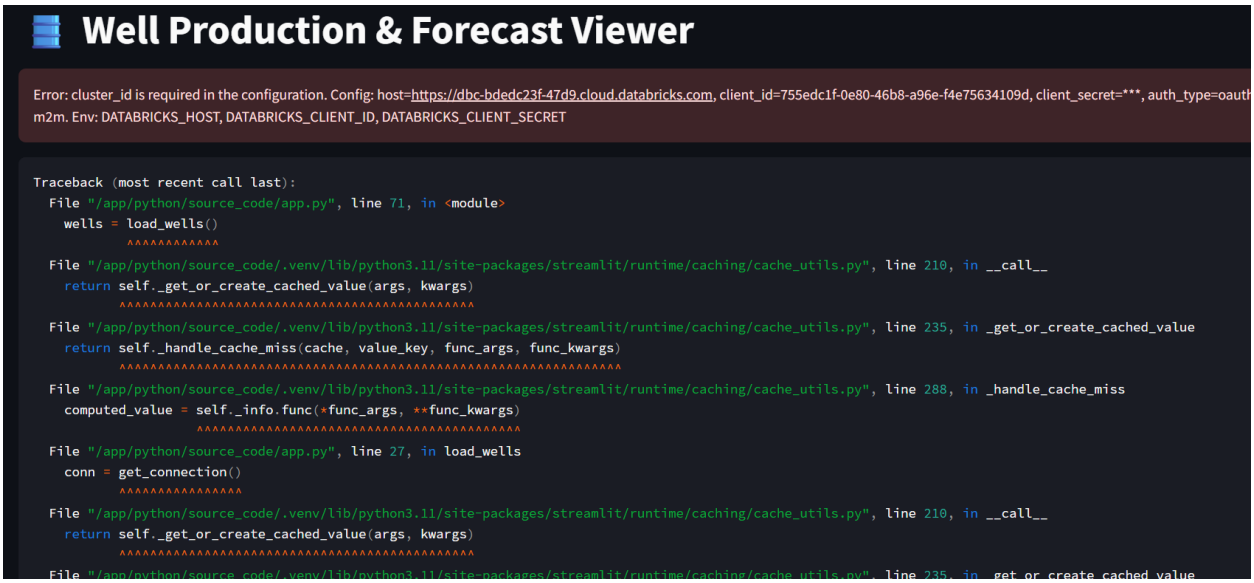
180 284.4394542905666

7.) Genie Code was very unhelpful and finicky, I tried for over 4 hours to get it to create this many different times. The closest I got to a working app looked like this:



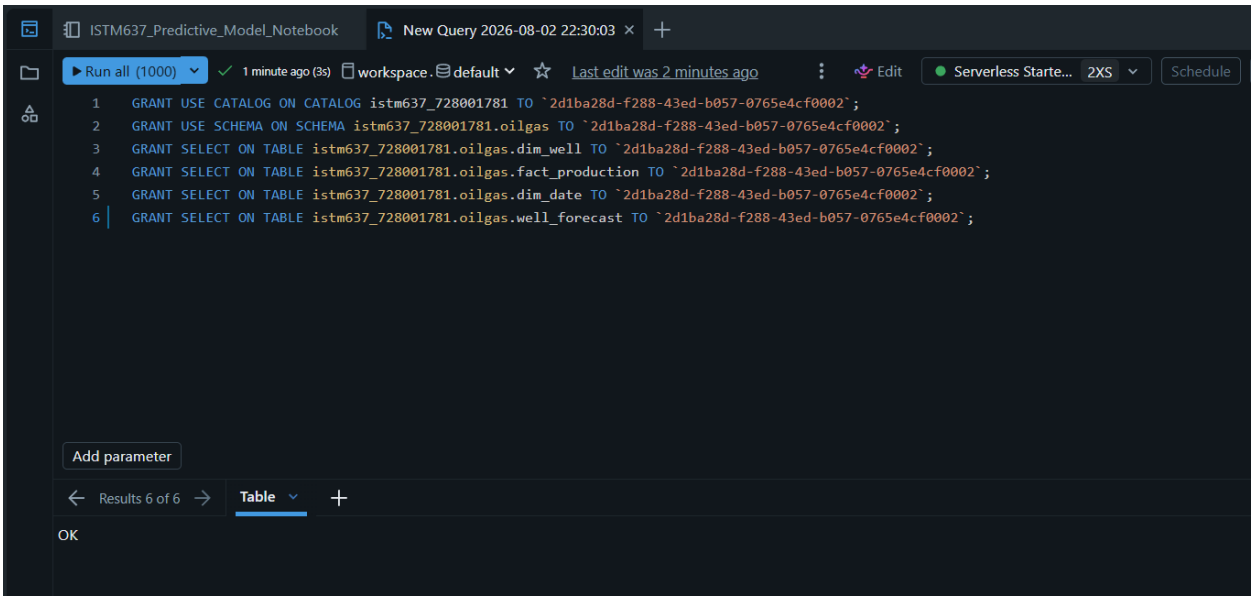
It simply would not give me forecasting, no matter what I tried.

Most of the time it would generate something like this:

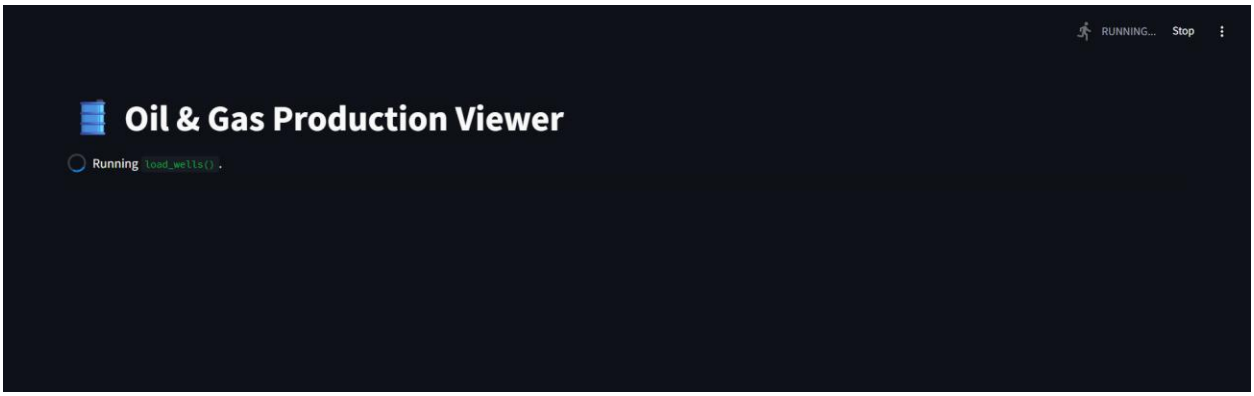


The final prompt I did to start fresh was this: Build a one-page Databricks App for oil & gas production using Unity Catalog tables. Use a dropdown that lists wells from istm637_728001781.oilgas.dim_well. When a well is selected, run exactly two SQL queries against my SQL warehouse: SELECT d.calendar_date, f.oil_bbl FROM istm637_728001781.oilgas.fact_production f JOIN istm637_728001781.oilgas.dim_date d ON f.date_id = d.date_id WHERE f.well_id = :well SELECT day_ahead, predicted_oil_bbl FROM istm637_728001781.oilgas.well_forecast WHERE well_id = :well Use the results to render two line charts on a single page: the first chart shows historical daily (or monthly-aggregated) oil_bbl over calendar_date for the selected well; the second chart shows the 180-day forecast curve from well_forecast for the same well. Keep the app simple and lightweight: one well selector, two charts, no model loading in the app (read the pre-computed well_forecast table instead of calling MLflow). Generate the app code (for example, using Streamlit or Dash) and configure it to run as a Databricks App with the right Unity Catalog table permissions so it can query these three tables.

It then requested I run this code, so I did, and got the output saying “ok”



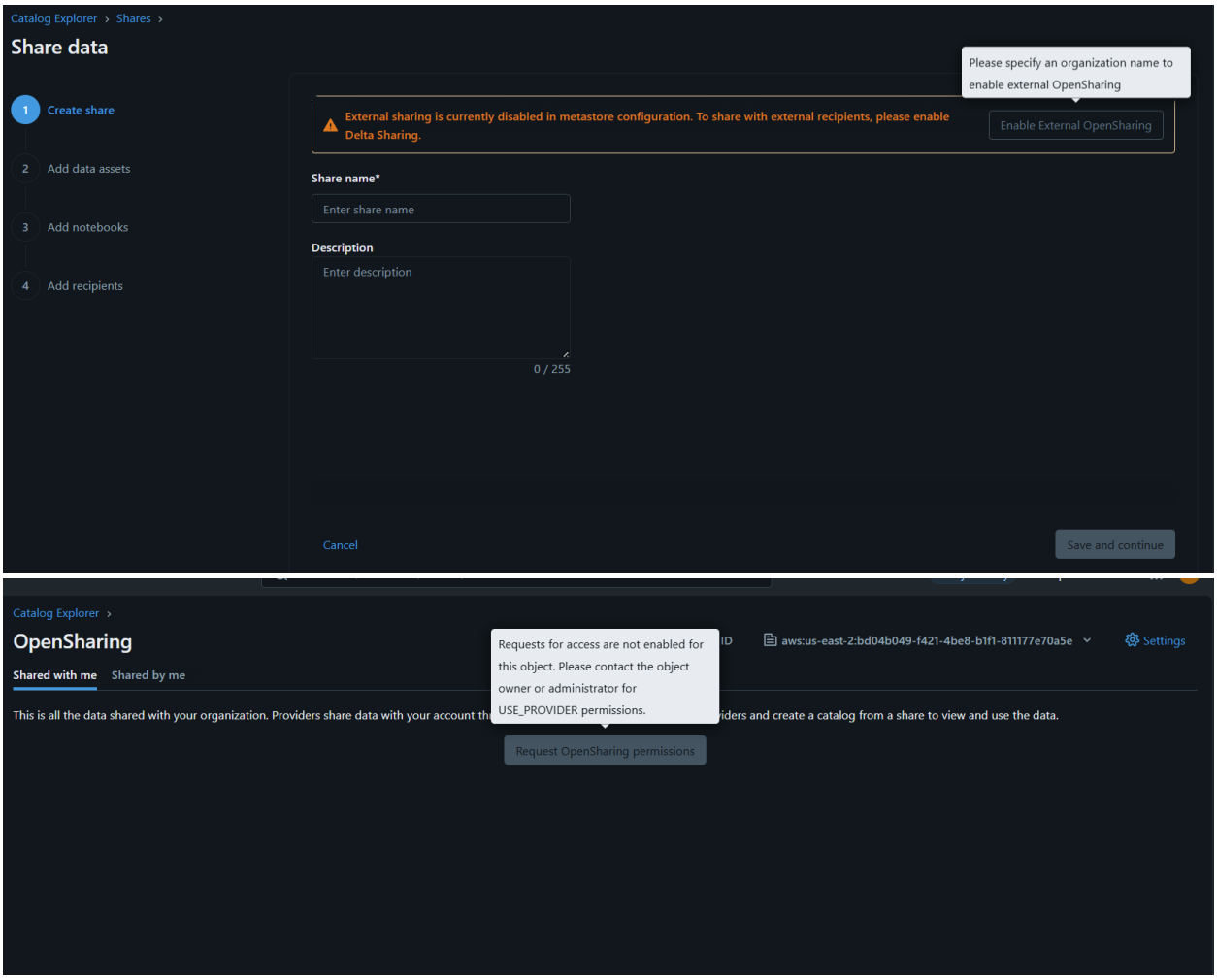
However, the app was now stuck at this point for over two hours.



Where the app data comes from: The app reads governed production data directly from Unity Catalog. Historical daily oil volumes come from the fact_production table, joined to dim_well for well attributes and dim_date for calendar labels and filters. The forecast curve is sourced from the well_forecast table, which is populated in the predictive model notebook using the registered oil_rate_predictor@champion model. Both charts in the app are driven by simple SQL queries against these tables, filtered by the selected well, so the app uses the same warehouse, schema, and governance as the Genie Agent and dashboard. This design keeps compute light by reusing the pre-computed well_forecast table instead of loading the model inside the app.

8.) When I tried to use OpenSharing for Part 8, the Databricks UI showed the message “External sharing is currently disabled in metastore configuration. To share with external

recipients, please enable Delta Sharing." This means my Unity Catalog metastore is not configured to allow external OpenSharing, and on Free Edition I don't have access to the account-level settings needed to turn this on. Instead of doing a real share with a partner, I followed the fallback instructions: I documented the normal provider workflow (create a share, add the dim_well table, generate a credential file, and let a non-Databricks recipient query it with the open Delta Sharing client) and included a screenshot of the disabled external sharing state. This demonstrates that I understand how Open Sharing would work in a fully enabled environment, even though my Free Edition workspace cannot complete the external share.



One Page Explanation:

In my workspace, external OpenSharing recipients are disabled, so instead of completing a full Databricks-to-Databricks exchange I implemented the documented fallback and focused on the provider workflow and Databricks-to-Open model. First, I created an

OpenSharing share (for example, dim_well_share) in Unity Catalog and added my governed dim_well table as the shared data asset, then captured a screenshot of the share with the table attached. In a fully enabled environment, the provider would next create a recipient object for a non-Databricks user with token-based authentication; Databricks would generate a bearer token, a credential profile file, and an activation link, and the provider would send that link to the recipient over a secure channel. The recipient would download the credential file (for example, config/dim_well_profile.share), save it locally, and then use an open-source Delta Sharing client to read the shared table via the open protocol. In Python they could run:

```
import delta_sharing

profile_file = "config/dim_well_profile.share"

client = delta_sharing.SharingClient(profile_file)

print(client.list_all_tables())

table_url = profile_file + "#dim_well_share.dim_well"

df = delta_sharing.load_as_pandas(table_url)
```

or in Spark:

```
table_path = "config/dim_well_profile.share#dim_well_share.dim_well"

df = spark.read.format("deltaSharing").load(table_path)

df.createOrReplaceTempView("shared_dim_well")

spark.sql("SELECT well_id, basin, operator FROM shared_dim_well LIMIT 10").show()
```

This shows that the provider exposes governed Unity Catalog tables through an OpenSharing share and a token-secured credential file, and that a non-Databricks recipient can consume the share using standard open-source clients and simple SQL/Python, even if they are not on Databricks, while my Free Edition workspace is limited to demonstrating the provider side of this workflow.